Amazon CloudWatch — Metrics, Logs, Alarms, Dashboards, and Events

Detailed Explanation

Overview: Unified Monitoring and Observability

Amazon CloudWatch is a comprehensive monitoring and observability service that provides a unified platform for collecting, tracking, and analyzing metrics, logs, traces, and events across your AWS infrastructure and applications[1]. It enables real-time visibility into system performance, application behavior, and operational health.

Core Components:

1. **Metrics**: Quantitative data points that measure system performance
2. **Logs**: Event records from applications, systems, and services
3. **Alarms**: Automated triggers that respond to metric conditions
4. **Dashboards**: Visual representations of metrics and logs
5. **Events**: Structured records of AWS resource state changes and application events
6. **Logs Insights**: Query engine for analyzing log data at scale
7. **Anomaly Detection**: Machine learning-based detection of unusual patterns

Amazon CloudWatch Metrics

What are CloudWatch Metrics?

CloudWatch Metrics are quantitative data points that represent the performance and behavior of your AWS resources and applications[2]. Each metric is identified by a namespace, metric name, and optional dimensions. Metrics are collected at regular intervals and are used to track system health, performance, and cost.

Key Characteristics:

- **Real-time Data**: Collected at intervals (1-minute standard, 1-second detailed)
- **Dimension Support**: Organize metrics with key-value pairs (e.g., InstanceId, Environment)
- **Retention**: Standard metrics stored for 15 months automatically
- **Resolution**: Standard (1-minute) or High-Resolution (1-second to 60-second)
- **Custom Metrics**: Push application-specific metrics using API/SDK
- **Metric Math**: Combine multiple metrics using mathematical operations
- **Statistics**: Aggregate data (Sum, Average, Maximum, Minimum, SampleCount)

CloudWatch Metrics Architecture:

AWS Resources / Applications
↓
Metric Collection
├─ AWS Resources (automatic)

```
│  ├── EC2 Instance CPU, Memory, Disk I/O
│  ├── RDS Database CPU, Connections, Query Performance
│  ├── Lambda Invocations, Duration, Errors
│  └── ELB Request Count, Latency
│
└── Custom Metrics (application push)
├── Business Metrics (orders, revenue)
├── Application Metrics (response time, queue depth)
└── Infrastructure Metrics (disk space, connections)
↓
Metric Storage (15 months retention)
↓
CloudWatch Console / API / Dashboards
↓
Alarms / Anomaly Detection / Analysis
```

Metric Namespaces and Dimensions

AWS Managed Namespaces:

Namespace	Resource Type	Example Metrics
AWS/EC2	EC2 Instances	CPUUtilization, NetworkIn, DiskReadOps
AWS/RDS	RDS Databases	CPUUtilization, DatabaseConnections, QueryLatency
AWS/Lambda	Lambda Functions	Invocations, Duration, Errors, Throttles
AWS/ApplicationELB	Load Balancers	TargetResponseTime, RequestCount, HealthyHostCount
AWS/S3	S3 Buckets	BucketSizeBytes, NumberOfObjects, 4xxErrors, 5xxErrors
AWS/DynamoDB	DynamoDB Tables	ConsumedWriteCapacity, ConsumedReadCapacity, UserErrors
AWS/SQS	SQS Queues	ApproximateNumberOfMessagesVisible, MessagesReceived

Dimensions Example (RDS):

```
{
"Namespace": "AWS/RDS",
"MetricName": "CPUUtilization",
```

"Dimensions": [
{
"Name": "DBInstanceIdentifier",
"Value": "production-database"
},
{
"Name": "DBClusterIdentifier",
"Value": "prod-cluster"
}
],
"Statistics": ["Average", "Maximum"],
"Period": 300 // 5 minutes
}

Metric Statistics

CloudWatch automatically aggregates raw metric data into statistics:

Statistic	Description	Use Case
Sum	Total of all values	Total requests, total bytes transferred
Average	Mean of all values	Average response time, average CPU utilization
Maximum	Highest value	Peak CPU, maximum response time
Minimum	Lowest value	Minimum response time
SampleCount	Number of data points	Request count over period
Percentile	P99, P95, P50 values	Latency percentiles for SLA tracking

Example: EC2 CPU Statistics (300-second period):

Raw Data Points (60 points per 5 minutes):
45%, 47%, 48%, 50%, 52%, 55%, 53%, 52%, 50%, 48%, 45%, 42%, 40%, 38%, 35%,
65%, 70%, 75%, 80%, 85%, 82%, 80%, 78%, 75%, 72%, 70%, 68%, 65%, 62%, 60%,
45%, 43%, 41%, 40%, 39%, 38%, 37%, 36%, 35%, 34%, 33%, 32%, 31%, 30%, 29%,
28%, 27%, 26%, 25%, 24%, 23%, 22%, 21%, 20%, 19%, 18%, 17%, 16%, 15%, 14%

Aggregated Statistics (5-minute period):
Average: 44.2%
Maximum: 85%
Minimum: 14%
Sum: 2,652
SampleCount: 60

Custom Metrics

You can publish custom metrics from your applications:

```
import boto3
from datetime import datetime

cloudwatch = boto3.client('cloudwatch')

# Put custom metric
cloudwatch.put_metric_data(
Namespace='MyApplication',
MetricData=[
{
'MetricName': 'OrderProcessingTime',
'Value': 125.5,
'Unit': 'Milliseconds',
'Timestamp': datetime.utcnow(),
'Dimensions': [
{'Name': 'Environment', 'Value': 'production'},
{'Name': 'Region', 'Value': 'us-east-1'}
]
},
{
'MetricName': 'QueueDepth',
'Value': 450,
'Unit': 'Count',
'Timestamp': datetime.utcnow(),
'Dimensions': [
{'Name': 'QueueName', 'Value': 'OrderQueue'}
]
}
]
)
```

Amazon CloudWatch Logs

What are CloudWatch Logs?

CloudWatch Logs is a fully managed service for collecting, monitoring, and storing log files from AWS resources, on-premises servers, and applications[3]. Logs are organized into log groups and log streams, enabling structured storage and querying of log data.

Key Characteristics:

- **Log Organization**: Hierarchical structure (log groups → log streams)
- **Real-time Monitoring**: View logs as they're generated
- **Log Retention**: Configurable retention (1 day to unlimited)
- **Log Filtering**: Pattern-based search and filtering
- **Metric Filters**: Extract metrics from log entries
- **Encryption**: Encrypt logs at rest using KMS

- **Costs**: Pay per GB ingested and stored (cheapest long-term storage)

Log Structure:

```
Log Groups (logical organization by application/service)
├── /aws/lambda/MyFunction
│   ├── Log Stream: 2024/01/15/[LATEST]defghij7890abcde
│   ├── Log Event: [2024-01-15T10:30:02Z] START RequestId: def456
│   └── ...
│
├── /aws/ecs/my-service
│   ├── Log Stream: container-123/app
│   ├── Log Stream: container-124/app
│   └── Log Stream: container-125/app
│
└── /aws/rds/error
    ├── Log Stream: database-1
    └── Log Stream: database-2
```

Log Groups and Log Streams

Log Group: A logical grouping of logs from a related source

Create log group

```
aws logs create-log-group
--log-group-name /myapp/production
```

Set retention (30 days)

```
aws logs put-retention-policy
--log-group-name /myapp/production
--retention-in-days 30
```

Log Stream: Individual sequence of log events from a single source

Create log stream (for on-premises agent)

```
aws logs create-log-stream
--log-group-name /myapp/production
--log-stream-name web-server-01
```

Put log events

```
aws logs put-log-events
--log-group-name /myapp/production
--log-stream-name web-server-01
--log-events timestamp=1234567890000,message="Application started"
```

Metric Filters

Extract metrics from log data to create custom CloudWatch metrics:

Create metric filter for errors

aws logs put-metric-filter
--log-group-name /myapp/production
--filter-name ErrorCount
--filter-pattern "[msg1, msg2, msg3, status_code = 5*, ...]"
--metric-transformations
metricName=ApplicationErrors,
metricNamespace=MyApp,
metricValue=1

Now can alarm on this metric

aws cloudwatch put-metric-alarm
--alarm-name HighErrorRate
--metric-name ApplicationErrors
--namespace MyApp
--statistic Sum
--period 300
--threshold 100
--comparison-operator GreaterThanThreshold

CloudWatch Logs Insights

Query and analyze logs at scale using SQL-like syntax[4]:

-- Find top 10 slowest API endpoints
fields @timestamp, @message, @duration
| filter ispresent(@duration)
| stats avg(@duration), max(@duration), count() by @message
| sort max(@duration) desc
| limit 10

-- Count errors by hour
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() as error_count by bin(5m)

-- Analyze Lambda performance
fields @duration, @billedDuration, @memoryUsed
| stats avg(@duration), max(@duration), avg(@memoryUsed) by bin(1m)

-- Track database connections
fields @timestamp, connection_count
| stats avg(connection_count) as avg_connections, max(connection_count) as peak_connections by bin(5m)

Amazon CloudWatch Alarms

What are CloudWatch Alarms?

CloudWatch Alarms continuously monitor metric values and execute actions when thresholds are exceeded[5]. Alarms are the primary mechanism for automated incident response in CloudWatch.

Alarm States:

Alarm Lifecycle:

Initial State: INSUFFICIENT_DATA
↓
Metric exceeds threshold → OK state
↓
Metric exceeds upper threshold → ALARM state (action triggered)
↓
Metric returns to normal → OK state (recovery action)
↓
No data for configured period → INSUFFICIENT_DATA state

Alarm Types:

Type	Description	Use Case
Threshold Alarm	Triggers when metric exceeds/falls below threshold	High CPU, Low Disk Space, High Latency
Anomaly Alarm	Uses machine learning to detect unusual patterns	Unexpected traffic spike, unusual error rate
Composite Alarm	Combines multiple alarms using AND/OR logic	Overall system health assessment
Alarms on Logs	Monitor log patterns	Track security events, application errors
Alarms on Metrics Insights	Dynamic alarms on complex queries	Monitor aggregate metrics across fleet

Creating and Managing Alarms

Threshold Alarm Example:

```
import boto3

cloudwatch = boto3.client('cloudwatch')

# Create alarm for high CPU

cloudwatch.put_metric_alarm(
AlarmName='HighCPUUtilization',
ComparisonOperator='GreaterThanThreshold',
EvaluationPeriods=2,
MetricName='CPUUtilization',
Namespace='AWS/EC2',
Period=300,
Statistic='Average',
Threshold=80,
ActionsEnabled=True,
AlarmActions=[
'arn:aws:sns:us-east-1:123456789012:alerts'
],
Dimensions=[
{'Name': 'InstanceId', 'Value': 'i-1234567890abcdef0'}
],
TreatMissingData='notBreaching',
Tags=[
{'Key': 'Environment', 'Value': 'production'}
]
)
```

Key Alarm Parameters:

Parameter	Description
EvaluationPeriods	Number of periods to evaluate before triggering (2 = needs 2 consecutive breaches)
Period	Duration of each evaluation period (seconds)
Threshold	Value to compare against metric
Statistic	How to aggregate data (Average, Sum, Maximum, Minimum)
TreatMissingData	Action when no data: notBreaching, missing, breaching
DatapointsToAlarm	Number of breaching datapoints needed to trigger (out of EvaluationPeriods)

Anomaly Alarm (Machine Learning):

Create anomaly alarm

```
cloudwatch.put_metric_alarm(
AlarmName='AnomalousLatency',
ComparisonOperator='LessThanLowerOrGreaterThanUpperThreshold',
EvaluationPeriods=1,
Metrics=[
{
'Id': 'm1',
'ReturnData': True,
'MetricStat': {
'Metric': {
'Namespace': 'AWS/ApplicationELB',
'MetricName': 'TargetResponseTime'
},
'Period': 300,
'Stat': 'Average'
}
},
{
'Id': 'ad1',
'Expression': 'ANOMALY_DETECTION_BAND(m1, 2)', # 2 standard deviations
'ReturnData': True
}
],
ThresholdMetricId='ad1'
)
```

Composite Alarm (Multiple Conditions):

Create composite alarm combining multiple conditions

```
cloudwatch.put_composite_alarm(
AlarmName='SystemUnhealthy',
AlarmRule='"
(ALARM(HighCPUUtilization) OR ALARM(HighMemoryUsage))
AND ALARM(HighDiskUtilization)
"',
ActionsEnabled=True,
AlarmActions=[
'arn:aws:sns:us-east-1:123456789012:critical-alerts'
]
)
```

Alarm Actions

Alarms can trigger automated responses:

SNS Notification:

```
cloudwatch.put_metric_alarm(
AlarmName='HighErrorRate',
# ... alarm configuration ...
AlarmActions=[
'arn:aws:sns:us-east-1:123456789012:alerts'
]
)
```

Auto Scaling Action:

```
cloudwatch.put_metric_alarm(
AlarmName='ScaleUp',
# ... alarm configuration ...
AlarmActions=[
'arn:aws:autoscaling:us-east-1:123456789012:ScaleUp'
]
)
```

Lambda Function:

Use EventBridge to trigger Lambda on alarm state change

(CloudWatch alarms send events to EventBridge)

Systems Manager OpsItems:

```
cloudwatch.put_metric_alarm(
AlarmName='DatabaseErrorRate',
# ... alarm configuration ...
AlarmActions=[
'arn:aws:ssm:us-east-1:123456789012:opsitem:1'
]
)
```

Amazon CloudWatch Dashboards

What are CloudWatch Dashboards?

CloudWatch Dashboards are customizable collections of widgets that visualize metrics, logs, and alarms in a unified view[6]. Dashboards provide real-time visibility into system health and enable quick identification of issues.

Dashboard Features:

- **Widget Types**: Graph, Number, Log Table, Metric Table, Alarm Status, Text
- **Customization**: Arrange widgets, set metrics, choose time ranges
- **Sharing**: Share dashboards across organization (read-only links)
- **Automatic Updates**: Enable auto-refresh or set refresh intervals
- **Multiple Metrics**: Single widget can display multiple metrics
- **Metric Math**: Display calculated metrics
- **Templating**: Create dynamic dashboards with variables

Dashboard Structure:

```
{
"widgets": [
{
"type": "metric",
"properties": {
"metrics": [
["AWS/EC2", "CPUUtilization", {"stat": "Average"}],
["AWS/EC2", "NetworkIn", {"stat": "Sum"}]
],
"period": 300,
"stat": "Average",
"region": "us-east-1",
"title": "EC2 Performance"
}
},
{
"type": "log",
"properties": {
```

"query": "fields @timestamp, @message | filter @message like /ERROR/",
"region": "us-east-1",
"title": "Application Errors"
}
},
{
"type": "alarm",
"properties": {
"alarms": ["arn:aws:cloudwatch:us-east-1:123456789012:alarm:HighCPU"]
}
}
]
}

Widget Types:

Type	Purpose	Use Case
Metric	Display metric graphs over time	System performance trends
Number	Show single current value	Current CPU%, Queue Depth
Log	Display log entries	Recent errors, application logs
Alarm	Show alarm status	System health overview
Metric Table	Tabular metric display	Compare multiple resources
Text	Add documentation	Runbook links, team contacts

Creating Dashboards

```
import json

cloudwatch = boto3.client('cloudwatch')

dashboard_body = {
"widgets": [
{
"type": "metric",
"properties": {
"metrics": [
["AWS/EC2", "CPUUtilization", {"InstanceId": "i-1234567890abcdef0"}],
["AWS/EC2", "NetworkIn", {"InstanceId": "i-1234567890abcdef0"}]
],
```

```
"period": 300,
"stat": "Average",
"region": "us-east-1",
"title": "EC2 Instance Performance",
"yAxis": {
"left": {
"label": "CPU %",
"min": 0,
"max": 100
}
}
}
},
{
"type": "number",
"properties": {
"metrics": [
["AWS/RDS", "DatabaseConnections", {"DBInstanceIdentifier": "prod-db"}]
],
"period": 300,
"stat": "Average",
"region": "us-east-1",
"title": "Current Database Connections"
}
}
]
}

cloudwatch.put_dashboard(
DashboardName='ProductionDashboard',
DashboardBody=json.dumps(dashboard_body)
)
```

Amazon CloudWatch Events and EventBridge Integration

CloudWatch Events and EventBridge

CloudWatch Events (now part of EventBridge) automatically captures AWS resource state changes and application events[7].

Event Types:

1. AWS API Events

```
{
"version": "0",
"id": "6a7e8feb-b491-4cf7-a9f1-bf3703467718",
"detail-type": "EC2 Instance State-change Notification",
"source": "aws.ec2",
"account": "123456789012",
"time": "2024-01-15T10:15:30Z",
"region": "us-east-1",
```

"resources": [
"arn:aws:ec2:us-east-1:123456789012:instance/i-1234567890abcdef0"
],
"detail": {
"instance-id": "i-1234567890abcdef0",
"state": "running"
}
}

2. CloudWatch Alarm State Change Events

{
"version": "0",
"id": "12345678-1234-1234-1234-123456789012",
"detail-type": "CloudWatch Alarm State Change",
"source": "aws.cloudwatch",
"account": "123456789012",
"time": "2024-01-15T10:30:00Z",
"region": "us-east-1",
"detail": {
"alarmName": "HighCPUUtilization",
"previousState": {
"value": "OK"
},
"state": {
"value": "ALARM"
}
}
}

3. Custom Application Events

```
import boto3
import json

eventbridge = boto3.client('events')

# Send custom event
eventbridge.put_events(
Entries=[
{
'Source': 'myapp',
'DetailType': 'UserAction',
'Detail': json.dumps({
'action': 'order_placed',
'order_id': '12345',
'amount': 99.99,
'customer_id': 'cust-789'
})
}
```

```
]
)
```

EventBridge Rules for Automation

Rule: Auto-respond to Failed EC2 Instance

```
eventbridge.put_rule(
Name='FailedEC2Recovery',
EventPattern=json.dumps({
'source': ['aws.ec2'],
'detail-type': ['EC2 Instance State-change Notification'],
'detail': {
'state': ['stopped', 'terminated']
}
}),
State='ENABLED'
)

eventbridge.put_targets(
Rule='FailedEC2Recovery',
Targets=[
{
'Arn': 'arn:aws:lambda:us-east-1:123456789012:function:RecoverEC2',
'Id': '1'
}
]
)
```

Rule: Alert on High CPU Alarms

```
eventbridge.put_rule(
Name='HighCPUAlert',
EventPattern=json.dumps({
'source': ['aws.cloudwatch'],
'detail-type': ['CloudWatch Alarm State Change'],
'detail': {
'alarmName': ['HighCPUUtilization'],
'state': {
'value': ['ALARM']
}
}
}),
State='ENABLED'
)

eventbridge.put_targets(
Rule='HighCPUAlert',
Targets=[
{
'Arn': 'arn:aws:sns:us-east-1:123456789012:critical-alerts',
'Id': '1'
```

```
}
]
)
```

Feature Comparison Matrix

Feature	Metrics	Logs	Alarms	Dashboards	Events
Real-time Visibility	✅	✅	✅	✅	✅
Automated Responses	❌	❌	✅	❌	✅
Data Analysis	✅	✅ (Insights)	❌	✅	❌
Historical Data	✅ (15 months)	✅ (configurable)	N/A	✅	✅ (limited)
Custom Dimensions	✅	❌	✅	✅	✅
Cost Optimization	Low	Medium	Low	Low	Low
Integration Points	High	Medium	High	Medium	Very High

Pricing Model

CloudWatch Metrics Pricing:

- Detailed Monitoring (1-min): $0.01 per metric per month for AWS resources
- Custom Metrics: $0.30 per custom metric per month
- API Requests: $0.01 per 1,000 requests

CloudWatch Logs Pricing:

- Ingestion: $0.50 per GB ingested
- Storage: $0.03 per GB stored per month
- Log Insights queries: $0.0048 per GB scanned

CloudWatch Alarms Pricing:

- Alarm: $0.10 per alarm per month (first 10 free)
- Composite Alarm: $0.10 per alarm per month
- Anomaly Alarm: $0.10 per alarm per month

Example Monthly Cost Calculation:

50 EC2 instances + 20 custom metrics + 100GB logs:

- Detailed EC2 metrics: 50 × $0.01 = $0.50
- Custom metrics: 20 × $0.30 = $6.00
- Log ingestion (assuming 20GB/month): 20 × $0.50 = $10.00
- Log storage (assuming 100GB): 100 × $0.03 = $3.00
- Alarms (100): (100-10) × $0.10 = $9.00
- **Total: ~$28.50/month**

Detailed Examples

Example 1: Comprehensive Application Monitoring with CloudWatch

Scenario: E-commerce application needs end-to-end monitoring including application metrics, system logs, and automated responses.

Step 1: Setup Application Metrics and Logs

import boto3
import json
from datetime import datetime
import time

class ApplicationMonitoring:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')
self.logs = boto3.client('logs')

```
def setup_log_groups(self):
    """Create log groups for application"""

    log_groups = [
        '/ecommerce/web-api',
        '/ecommerce/payment-service',
        '/ecommerce/order-processing',
        '/ecommerce/security'
    ]

    for log_group in log_groups:
        try:
            self.logs.create_log_group(logGroupName=log_group)
            self.logs.put_retention_policy(
                logGroupName=log_group,
```

```
                retentionInDays=30
            )
            print(f"Created log group: {log_group}")
        except self.logs.exceptions.ResourceAlreadyExistsException:
            print(f"Log group already exists: {log_group}")

def setup_metric_filters(self):
    """Create metric filters to extract metrics from logs"""

    # Error count metric filter
    self.logs.put_metric_filter(
        logGroupName='/ecommerce/web-api',
        filterName='ErrorCount',
        filterPattern='[timestamp, request_id, level = ERROR, ...]',
        metricTransformations=[
            {
                'metricName': 'ApplicationErrors',
                'metricNamespace': 'Ecommerce/WebAPI',
                'metricValue': '1',
                'defaultValue': 0
            }
        ]
    )

    # Slow request metric filter
    self.logs.put_metric_filter(
        logGroupName='/ecommerce/web-api',
        filterName='SlowRequests',
        filterPattern='[timestamp, request_id, duration > 1000, ...]',
        metricTransformations=[
            {
                'metricName': 'SlowRequests',
                'metricNamespace': 'Ecommerce/WebAPI',
                'metricValue': '1',
                'defaultValue': 0
            }
        ]
    )
```

```
    print("Metric filters created")

def publish_custom_metrics(self):
    """Publish application metrics"""

    metrics = [
        {
            'MetricName': 'OrdersProcessed',
            'Value': 1250,
            'Unit': 'Count',
            'Namespace': 'Ecommerce/Orders',
            'Dimensions': [
                {'Name': 'Environment', 'Value': 'production'},
                {'Name': 'Region', 'Value': 'us-east-1'}
            ]
        },
        {
            'MetricName': 'AverageOrderValue',
            'Value': 85.50,
            'Unit': 'None',
            'Namespace': 'Ecommerce/Orders',
            'Dimensions': [
                {'Name': 'Environment', 'Value': 'production'}
            ]
        },
        {
            'MetricName': 'CheckoutConversionRate',
            'Value': 78.5,
            'Unit': 'Percent',
            'Namespace': 'Ecommerce/Sales',
            'Dimensions': [
                {'Name': 'Environment', 'Value': 'production'}
            ]
        },
        {
            'MetricName': 'PaymentGatewayLatency',
            'Value': 245,
```

```
            'Unit': 'Milliseconds',
            'Namespace': 'Ecommerce/Payments',
            'Dimensions': [
                {'Name': 'PaymentProvider', 'Value': 'Stripe'}
            ]
        }
    ]

    self.cloudwatch.put_metric_data(
        Namespace='Ecommerce',
        MetricData=metrics
    )

    print(f"Published {len(metrics)} metrics")

def simulate_application_logs(self):
    """Simulate application logging"""

    log_events = [
        {
            'logGroupName': '/ecommerce/web-api',
            'logStreamName': 'web-server-01',
            'events': [
                {'message': '[2024-01-15 10:30:00] INFO Request GET /api/products durat
                {'message': '[2024-01-15 10:30:01] INFO Request GET /api/cart duration=
                {'message': '[2024-01-15 10:30:02] ERROR Request POST /api/checkout fa
                {'message': '[2024-01-15 10:30:03] INFO Request GET /api/products durat
            ]
        }
    ]

    # Create log stream and put events
    try:
        self.logs.create_log_stream(
            logGroupName='/ecommerce/web-api',
            logStreamName='web-server-01'
        )
    except self.logs.exceptions.ResourceAlreadyExistsException:
```

```
            pass

        for log_event in log_events:
            events = [
                {
                    'timestamp': int(time.time() * 1000),
                    'message': event['message']
                }
                for event in log_event['events']
            ]

            self.logs.put_log_events(
                logGroupName=log_event['logGroupName'],
                logStreamName=log_event['logStreamName'],
                logEvents=events
            )

        print("Application logs simulated")
```

Usage

app_monitoring = ApplicationMonitoring()
app_monitoring.setup_log_groups()
app_monitoring.setup_metric_filters()
app_monitoring.publish_custom_metrics()
app_monitoring.simulate_application_logs()

Step 2: Create Alarms for Critical Metrics

class ApplicationAlarms:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')

```
    def create_application_alarms(self):
        """Create comprehensive alarm set"""

        alarms = [
            {
                'AlarmName': 'HighErrorRate',
                'MetricName': 'ApplicationErrors',
```

```
        'Namespace': 'Ecommerce/WebAPI',
        'Statistic': 'Sum',
        'Period': 300,
        'EvaluationPeriods': 2,
        'Threshold': 50,
        'ComparisonOperator': 'GreaterThanThreshold',
        'AlarmDescription': 'Alert when error count exceeds 50 in 5 minutes'
    },
    {
        'AlarmName': 'SlowCheckoutRequests',
        'MetricName': 'SlowRequests',
        'Namespace': 'Ecommerce/WebAPI',
        'Statistic': 'Sum',
        'Period': 300,
        'EvaluationPeriods': 1,
        'Threshold': 10,
        'ComparisonOperator': 'GreaterThanThreshold',
        'AlarmDescription': 'Alert on checkout latency issues'
    },
    {
        'AlarmName': 'PaymentGatewayDown',
        'MetricName': 'PaymentGatewayLatency',
        'Namespace': 'Ecommerce/Payments',
        'Statistic': 'Average',
        'Period': 60,
        'EvaluationPeriods': 3,
        'Threshold': 5000,  # >5 seconds
        'ComparisonOperator': 'GreaterThanThreshold',
        'TreatMissingData': 'breaching'
    },
    {
        'AlarmName': 'LowConversionRate',
        'MetricName': 'CheckoutConversionRate',
        'Namespace': 'Ecommerce/Sales',
        'Statistic': 'Average',
        'Period': 3600,  # 1 hour
        'EvaluationPeriods': 2,
        'Threshold': 70,
```

```
            'ComparisonOperator': 'LessThanThreshold',
            'AlarmDescription': 'Alert when conversion drops below 70%'
        }
    ]

    for alarm in alarms:
        self.cloudwatch.put_metric_alarm(
            AlarmName=alarm['AlarmName'],
            MetricName=alarm['MetricName'],
            Namespace=alarm['Namespace'],
            Statistic=alarm['Statistic'],
            Period=alarm['Period'],
            EvaluationPeriods=alarm['EvaluationPeriods'],
            Threshold=alarm['Threshold'],
            ComparisonOperator=alarm['ComparisonOperator'],
            AlarmActions=[
                'arn:aws:sns:us-east-1:123456789012:application-alerts'
            ],
            TreatMissingData=alarm.get('TreatMissingData', 'notBreaching'),
            AlarmDescription=alarm.get('AlarmDescription', '')
        )

        print(f"Created alarm: {alarm['AlarmName']}")
```

Usage

alarms = ApplicationAlarms()
alarms.create_application_alarms()

Step 3: Create Comprehensive Dashboard

import json

class ApplicationDashboard:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')

```
  def create_dashboard(self):
      """Create application monitoring dashboard"""
```

```
    dashboard_body = {
        'widgets': [
            # Row 1: Key Metrics
            {
                'type': 'metric',
                'properties': {
                    'metrics': [
                        ['Ecommerce/Orders', 'OrdersProcessed', {'stat': 'Sum'}],
                        ['Ecommerce/Sales', 'CheckoutConversionRate', {'stat': 'Average'}],
                        ['Ecommerce/Orders', 'AverageOrderValue', {'stat': 'Average'}]
                    ],
                    'period': 300,
                    'stat': 'Average',
                    'region': 'us-east-1',
                    'title': 'Business Metrics',
                    'yAxis': {'left': {'min': 0}}
                }
            },
            # Row 2: System Health
            {
                'type': 'metric',
                'properties': {
                    'metrics': [
                        ['Ecommerce/WebAPI', 'ApplicationErrors', {'stat': 'Sum'}],
                        ['Ecommerce/WebAPI', 'SlowRequests', {'stat': 'Sum'}],
                        ['Ecommerce/Payments', 'PaymentGatewayLatency', {'stat': 'Average
                    ],
                    'period': 300,
                    'stat': 'Average',
                    'region': 'us-east-1',
                    'title': 'System Health',
                    'yAxis': {'left': {'min': 0}}
                }
            },
            # Row 3: Alarms
            {
                'type': 'alarm',
                'properties': {
```

```
                'alarms': [
                    'arn:aws:cloudwatch:us-east-1:123456789012:alarm:HighErrorRate
                    'arn:aws:cloudwatch:us-east-1:123456789012:alarm:SlowCheckout
                    'arn:aws:cloudwatch:us-east-1:123456789012:alarm:PaymentGatew
                ],
                'title': 'Alarm Status'
            }
        },
        # Row 4: Recent Errors
        {
            'type': 'log',
            'properties': {
                'query': '''
                    fields @timestamp, @message, @logStream
                    | filter @message like /ERROR/
                    | stats count() as error_count by @logStream
                ''',
                'region': 'us-east-1',
                'title': 'Recent Errors by Service'
            }
        }
    ]
}

self.cloudwatch.put_dashboard(
    DashboardName='EcommerceApplicationDashboard',
    DashboardBody=json.dumps(dashboard_body)
)

print("Dashboard created: EcommerceApplicationDashboard")
```

Usage

dashboard = ApplicationDashboard()
dashboard.create_dashboard()

Example 2: Advanced Alarm Strategy with Composite and Anomaly Alarms

Scenario: Production system needs sophisticated alerting combining multiple conditions and anomaly detection.

class AdvancedAlarmStrategy:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')

```
def create_anomaly_detection_alarms(self):
    """Create ML-based anomaly detection alarms"""

    # Anomaly alarm for network traffic
    self.cloudwatch.put_metric_alarm(
        AlarmName='AnomalousNetworkTraffic',
        ComparisonOperator='LessThanLowerOrGreaterThanUpperThreshold',
        EvaluationPeriods=1,
        Metrics=[
            {
                'Id': 'm1',
                'ReturnData': True,
                'MetricStat': {
                    'Metric': {
                        'Namespace': 'AWS/EC2',
                        'MetricName': 'NetworkIn',
                        'Dimensions': [
                            {'Name': 'InstanceId', 'Value': 'i-1234567890abcdef0'}
                        ]
                    },
                    'Period': 300,
                    'Stat': 'Average'
                }
            },
            {
                'Id': 'ad1',
                'Expression': 'ANOMALY_DETECTION_BAND(m1, 2)',  # 2 std devs
                'ReturnData': True
            }
        ],
        ThresholdMetricId='ad1',
```

```
        AlarmActions=['arn:aws:sns:us-east-1:123456789012:alerts']
    )

    print("Created anomaly detection alarm")

def create_composite_alarms(self):
    """Create composite alarms combining multiple conditions"""

    # System Health composite alarm
    self.cloudwatch.put_composite_alarm(
        AlarmName='SystemUnhealthy',
        AlarmRule='''
            (ALARM(HighCPUUtilization) OR ALARM(HighMemoryUsage))
            AND
            (ALARM(HighDiskUtilization) OR ALARM(HighNetworkLatency))
        ''',
        ActionsEnabled=True,
        AlarmActions=['arn:aws:sns:us-east-1:123456789012:critical-alerts'],
        AlarmDescription='Triggered when multiple system health indicators are c
    )

    # Database Health composite alarm
    self.cloudwatch.put_composite_alarm(
        AlarmName='DatabaseUnhealthy',
        AlarmRule='''
            (ALARM(HighDatabaseCPU) OR ALARM(HighConnectionCount))
            AND
            (ALARM(HighReadLatency) OR ALARM(HighWriteLatency))
        ''',
        ActionsEnabled=True,
        AlarmActions=['arn:aws:sns:us-east-1:123456789012:db-alerts']
    )

    # Application Health composite alarm
    self.cloudwatch.put_composite_alarm(
        AlarmName='ApplicationUnhealthy',
        AlarmRule='''
            (ALARM(HighErrorRate) AND ALARM(HighResponseTime))
```

```
            OR
            ALARM(HighExceptionCount)
        ''',
        ActionsEnabled=True,
        AlarmActions=['arn:aws:sns:us-east-1:123456789012:app-alerts']
    )

    print("Created composite alarms")

def create_metric_math_alarms(self):
    """Create alarms using metric math"""

    # Error rate alarm using metric math
    self.cloudwatch.put_metric_alarm(
        AlarmName='ErrorRateAlarm',
        Metrics=[
            {
                'Id': 'errors',
                'ReturnData': False,
                'MetricStat': {
                    'Metric': {
                        'Namespace': 'Ecommerce',
                        'MetricName': 'ApplicationErrors'
                    },
                    'Period': 300,
                    'Stat': 'Sum'
                }
            },
            {
                'Id': 'requests',
                'ReturnData': False,
                'MetricStat': {
                    'Metric': {
                        'Namespace': 'Ecommerce',
                        'MetricName': 'TotalRequests'
                    },
                    'Period': 300,
                    'Stat': 'Sum'
```

```
            }
        },
        {
            'Id': 'error_rate',
            'Expression': '(errors / requests) * 100',
            'ReturnData': True
        }
    ],
    EvaluationPeriods=2,
    Threshold=5,
    ComparisonOperator='GreaterThanThreshold',
    ThresholdMetricId='error_rate',
    AlarmActions=['arn:aws:sns:us-east-1:123456789012:alerts']
)

print("Created metric math alarm")
```

Usage

advanced_alarms = AdvancedAlarmStrategy()
advanced_alarms.create_anomaly_detection_alarms()
advanced_alarms.create_composite_alarms()
advanced_alarms.create_metric_math_alarms()

Example 3: EventBridge Integration for Automated Responses

Scenario: Automatically respond to CloudWatch alarms and AWS events using EventBridge and Lambda.

import json

class EventBridgeAutomation:
def **init**(self):
self.eventbridge = boto3.client('events')
self.logs = boto3.client('logs')

```
def setup_cloudwatch_alarm_events(self):
    """Setup EventBridge rule to capture CloudWatch alarm state changes"""

    # Create EventBridge rule for alarm state changes
    self.eventbridge.put_rule(
```

```
        Name='CaptureCloudWatchAlarmStateChanges',
        EventPattern=json.dumps({
            'source': ['aws.cloudwatch'],
            'detail-type': ['CloudWatch Alarm State Change'],
            'detail': {
                'state': {
                    'value': ['ALARM', 'OK']
                }
            }
        }),
        State='ENABLED'
    )

    # Create log group for debugging
    log_group = '/eventbridge/alarm-events'
    try:
        self.logs.create_log_group(logGroupName=log_group)
    except self.logs.exceptions.ResourceAlreadyExistsException:
        pass

    # Add CloudWatch Logs as target for debugging
    self.eventbridge.put_targets(
        Rule='CaptureCloudWatchAlarmStateChanges',
        Targets=[
            {
                'Arn': f'arn:aws:logs:us-east-1:123456789012:log-group:{log_group}',
                'Id': '1',
                'RoleArn': 'arn:aws:iam::123456789012:role/EventBridgeRole'
            }
        ]
    )

    print("Setup alarm state change capture")

def setup_auto_scaling_rule(self):
    """Auto-scale when high CPU alarm triggers"""

    self.eventbridge.put_rule(
```

```
        Name='AutoScaleOnHighCPU',
        EventPattern=json.dumps({
            'source': ['aws.cloudwatch'],
            'detail-type': ['CloudWatch Alarm State Change'],
            'detail': {
                'alarmName': ['HighCPUUtilization'],
                'state': {
                    'value': ['ALARM']
                }
            }
        }),
        State='ENABLED'
    )

    # Trigger Lambda for scaling
    self.eventbridge.put_targets(
        Rule='AutoScaleOnHighCPU',
        Targets=[
            {
                'Arn': 'arn:aws:lambda:us-east-1:123456789012:function:ScaleUpASG',
                'Id': '1',
                'RoleArn': 'arn:aws:iam::123456789012:role/EventBridgeRole'
            }
        ]
    )

    print("Setup auto-scaling rule")

def setup_incident_creation_rule(self):
    """Create incident when critical alarm triggers"""

    self.eventbridge.put_rule(
        Name='CreateIncidentOnCriticalAlarm',
        EventPattern=json.dumps({
            'source': ['aws.cloudwatch'],
            'detail-type': ['CloudWatch Alarm State Change'],
            'detail': {
                'alarmName': [
```

```
                'DatabaseUnhealthy',
                'ApplicationUnhealthy',
                'SystemUnhealthy'
            ],
            'state': {
                'value': ['ALARM']
            }
        }
    }),
    State='ENABLED'
)

# Create incident via Systems Manager
self.eventbridge.put_targets(
    Rule='CreateIncidentOnCriticalAlarm',
    Targets=[
        {
            'Arn': 'arn:aws:ssm:us-east-1:123456789012:incident-response-plan/criti
            'Id': '1',
            'RoleArn': 'arn:aws:iam::123456789012:role/EventBridgeRole'
        }
    ]
)

print("Setup incident creation rule")

def setup_custom_event_rule(self):
    """Route custom application events"""

    self.eventbridge.put_rule(
        Name='RouteApplicationEvents',
        EventPattern=json.dumps({
            'source': ['myapp'],
            'detail-type': ['Order', 'Payment', 'Delivery']
        }),
        State='ENABLED'
    )
```

```
    # Route to different targets based on event type
    self.eventbridge.put_targets(
        Rule='RouteApplicationEvents',
        Targets=[
            {
                'Arn': 'arn:aws:sqs:us-east-1:123456789012:order-queue',
                'Id': '1',
                'RoleArn': 'arn:aws:iam::123456789012:role/EventBridgeRole'
            }
        ]
    )

    print("Setup custom event routing")
```

Usage

eventbridge_automation = EventBridgeAutomation()
eventbridge_automation.setup_cloudwatch_alarm_events()
eventbridge_automation.setup_auto_scaling_rule()
eventbridge_automation.setup_incident_creation_rule()
eventbridge_automation.setup_custom_event_rule()

Lambda Function for Automated Response:

import boto3
import json

autoscaling = boto3.client('autoscaling')
sns = boto3.client('sns')

def lambda_handler(event, context):
"""Handle CloudWatch alarm state change and trigger scaling"""

```
    detail = event['detail']
    alarm_name = detail.get('alarmName')
    state = detail['state']['value']

    print(f"Alarm: {alarm_name}, State: {state}")

    if alarm_name == 'HighCPUUtilization' and state == 'ALARM':
        # Scale up
        response = autoscaling.set_desired_capacity(
```

```
            AutoScalingGroupName='production-asg',
            DesiredCapacity=5,
            HonorCooldown=True
        )

        # Notify team
        sns.publish(
            TopicArn='arn:aws:sns:us-east-1:123456789012:alerts',
            Subject=f'Auto-scaling triggered: {alarm_name}',
            Message=f'''
            Alarm: {alarm_name}
            State: {state}
            Action: Scaled up to 5 instances
            Timestamp: {detail.get('state').get('timestamp')}
            '''
        )

        return {
            'statusCode': 200,
            'body': json.dumps('Auto-scaling triggered successfully')
        }

    return {
        'statusCode': 200,
        'body': json.dumps('No action taken')
    }
```

Top 5 Interview Questions

Question 1: Explain CloudWatch Metrics vs Logs — When Would You Use Each?

Scenario: "Your application needs complete observability. Explain the difference between CloudWatch Metrics and Logs, their use cases, and how they work together."

Answer Structure:

Fundamental Difference:

CloudWatch Metrics:

- Quantitative data points (numbers)
- Time-series data stored at regular intervals
- Examples: CPU%, Memory%, Request Count, Error Count
- Queryable via MetricStat (Sum, Average, Maximum, Minimum)
- Fast queries (optimized for time-series)
- 15-month retention (automatic)
- Cost-effective for aggregated data

CloudWatch Logs:

- Qualitative text data (strings with details)
- Complete event records with context
- Examples: Application logs, system events, API request details
- Queryable via Logs Insights (SQL-like language)
- Slower queries (full-text search)
- Configurable retention (1 day to unlimited)
- More expensive for high volume, but better for detailed analysis

Comparison Matrix:

Aspect	Metrics	Logs
Data Type	Numeric (time-series)	Text (events)
Granularity	Aggregated (1-min standard)	Individual events (millisecond precision)
Query Type	Statistical (Sum, Avg, Max)	Pattern matching (SQL-like)
Typical Use	Trending, thresholds, alarms	Investigation, debugging, audit
Cost	Low for long-term storage	Higher for high-volume ingestion
Real-time	✅ Yes	✅ Yes
Historical Analysis	✅ 15 months	✅ Configurable
Alarm Support	✅ Yes	Limited (via metric filters)

When to Use Metrics:

1. **System Performance Trending**
   - CPU, Memory, Disk utilization over time
   - Request rate changes

    - Error rate trends
2. **Setting Alarms**
    - Alert when CPU > 80%
    - Alert when error rate > 5%
    - Auto-scaling triggers
3. **Dashboard Visualization**
    - Quick visual overview of health
    - Historical trend analysis
    - Multi-resource comparison
4. **Business Analytics**
    - Orders per minute
    - Revenue trends
    - Conversion rate over time

When to Use Logs:

1. **Incident Investigation**
    - "Why did the request fail?"
    - Stack traces and error details
    - Request parameters for debugging
2. **Compliance and Audit**
    - Who accessed what and when
    - Security event details
    - Complete audit trail
3. **Application Debugging**
    - Find specific error messages
    - Trace request flow
    - Identify performance bottlenecks
4. **Complex Queries**
    - "All requests that took > 5 seconds AND had status 500"
    - "All failed requests from specific IP address"
    - Pattern extraction

Integration:

Application → Logs + Metrics
↓
Logs Insights Queries
Extract patterns → Create Metric Filters
↓
Derived Metrics
↓
Alarms & Dashboards

Practical Example:

Scenario: Application has high error rate

Using Metrics Only:

- See error count is 500/hour (metric)
- Know there's a problem
- Can't determine root cause

- Can set alert: "If errors > 400/hour"

Using Logs Only:

- See individual error messages
- Can debug specific failures
- Time-consuming to find patterns
- Need to manually count errors for trending

Using Both (Best Practice):

- Metrics show error rate is abnormally high
- Metrics trigger alarm
- Review logs to find root cause
- Extract new metric filter for specific error type
- Update alarm thresholds based on patterns
- Dashboard shows both trend and recent examples

Cost Consideration:

High-volume application (1M requests/hour):

Metrics Only:

- 10 custom metrics × $0.30/month = $3
- API requests: minimal
- Can't drill down to details
- Total: ~$5/month

Logs (Raw):

- 1M requests = ~50MB/hour = ~36GB/month
- Ingestion: 36GB × $0.50 = $18
- Storage: 36GB × $0.03 = $1.08
- Total: ~$20/month

Recommended Hybrid:

- Metrics for all key indicators: $5
- Sample logs (1%) for debugging: $0.20
- Metric filters for derived metrics: $0
- Total: ~$6/month (more insight than metrics alone)

Question 2: Design a Comprehensive Alarm Strategy for a Mission-Critical Application

Scenario: "Design a complete alarm strategy for a mission-critical microservices application with multiple tiers. Include threshold selection, composite alarms, and escalation procedures."

Answer Structure:

Alarm Strategy Framework:

Alarm Pyramid (by severity):

```
        CRITICAL
    (Page on-call immediately)
        3-4 alarms
           ↑
         MAJOR
    (Urgent team notification)
        8-10 alarms
           ↑
        MINOR
  (Alert for analysis/trends)
        15-20 alarms
```

Tier 1: Critical Infrastructure Alarms

```
critical_alarms = [
{
'name': 'DatabaseDown',
'metric': 'DatabaseConnections',
'namespace': 'AWS/RDS',
'threshold': 0,
'operator': 'LessThanThreshold',
'period': 60,
'evaluation_periods': 1,
'treat_missing_data': 'breaching',
'action': 'Page on-call immediately',
'escalation': '5 min if not acknowledged'
},
{
'name': 'APIGatewayDown',
'metric': 'Count',
'namespace': 'AWS/ApiGateway',
'threshold': 0,
'operator': 'LessThanThreshold',
'period': 60,
'evaluation_periods': 2,
'treat_missing_data': 'breaching',
'action': 'Page on-call + auto-failover',
'escalation': '2 min escalation'
},
{
'name': 'AuthServiceFailed',
'metric': '5XXError',
'namespace': 'Microservices',
'threshold': 100,
```

'operator': 'GreaterThanThreshold',
'period': 60,
'evaluation_periods': 2,
'action': 'Page on-call',
'escalation': '5 min'
}
]

Tier 2: Major Service Alarms

major_alarms = [
{
'name': 'HighDatabaseConnections',
'metric': 'DatabaseConnections',
'threshold': 450, # 90% of max 500
'operator': 'GreaterThanThreshold',
'statistic': 'Average',
'period': 300,
'evaluation_periods': 2,
'datapoints_to_alarm': 2,
'action': 'Notify team, consider connection pooling review'
},
{
'name': 'SlowDatabaseQueries',
'metric': 'QueryLatency',
'threshold': 2000, # >2 seconds
'operator': 'GreaterThanThreshold',
'statistic': 'P99',
'period': 300,
'evaluation_periods': 2,
'action': 'Alert team for query optimization'
},
{
'name': 'HighAPILatency',
'metric': 'ResponseTime',
'threshold': 500, # >500ms
'operator': 'GreaterThanThreshold',
'statistic': 'P95',
'period': 300,
'evaluation_periods': 3,
'action': 'Alert for performance investigation'
},
{
'name': 'HighErrorRate',
'metric': 'ErrorRate',
'threshold': 2, # >2% errors
'operator': 'GreaterThanThreshold',
'statistic': 'Average',
'period': 300,
'evaluation_periods': 2,
'action': 'Alert team, check logs for root cause'

}
]

Tier 3: Minor Operational Alarms

minor_alarms = [
{
'name': 'HighCPUUtilization',
'metric': 'CPUUtilization',
'threshold': 75,
'operator': 'GreaterThanThreshold',
'action': 'Monitor for scaling needs',
'evaluation_periods': 5 # Allow more time
},
{
'name': 'DiskSpaceWarning',
'metric': 'DiskUsagePercent',
'threshold': 85,
'operator': 'GreaterThanThreshold',
'action': 'Scheduled maintenance notification'
},
{
'name': 'UnusualNetworkTraffic',
'metric': 'NetworkIn',
'detection_type': 'ANOMALY',
'std_deviations': 2,
'action': 'Alert for investigation'
}
]

Threshold Selection Methodology:

For Performance Metrics:

Step 1: Establish Baseline

- Collect 1-2 weeks of data
- Identify normal range
- Calculate percentiles (P50, P95, P99)

Step 2: Define SLO (Service Level Objective)

- P95 latency: should be < 500ms
- Error rate: should be < 1%
- Availability: should be > 99.9%

Step 3: Set Alarm Threshold

- Warning threshold: 75% of SLO limit
- Critical threshold: 90% of SLO limit
- Example:
    - SLO P95 latency: 500ms
    - Warning alarm: 375ms (75%)

    - Critical alarm: 450ms (90%)

Step 4: Tune Based on False Positives

- Start conservative (high thresholds)
- Monitor alert frequency
- Lower if legitimate incidents missed
- Raise if false positives excessive

Composite Alarm Strategy:

System Health Indicator

```
composite_alarms = [
{
'name': 'SystemHealthDegraded',
'rule': '''
(ALARM(HighCPUUtilization) OR ALARM(HighMemoryUsage))
AND
(ALARM(HighNetworkLatency) OR ALARM(SlowDatabaseQueries))
''',
'action': 'Investigate infrastructure health'
},
{
'name': 'DataLayerFailure',
'rule': '''
ALARM(DatabaseDown)
OR
(ALARM(HighDatabaseConnections) AND ALARM(SlowDatabaseQueries))
''',
'action': 'Critical incident - immediate escalation'
},
{
'name': 'ApplicationCrash',
'rule': '''
ALARM(APIEndpointDown)
AND
(ALARM(HighErrorRate) OR ALARM(ResponseTimeout))
''',
'action': 'Page on-call + auto-remediation'
}
]
```

Escalation Procedures:

```
Level 1: Automated Response (0-1 minute)
├─ Trigger CloudWatch alarm
├─ Send SNS notification
└─ Execute auto-remediation (if applicable)
└─ Examples: Restart service, scale up, failover
```

Level 2: Team Notification (1-5 minutes)
├── Notify team via Slack
├── Create ticket in incident management
└── If not acknowledged → escalate

Level 3: Escalation (5-15 minutes)
├── Page on-call engineer
├── Create war room
└── If not responding → escalate to manager

Level 4: Crisis (15+ minutes)
├── Page manager and architect
├── Activate disaster recovery
├── Customer communication team
└── Executive notification if needed

Implementation:

class ComprehensiveAlarmStrategy:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')
self.sns = boto3.client('sns')

```
def create_alarm_hierarchy(self):
    """Create stratified alarm system"""

    # Tier 1: Critical - Page immediately
    self._create_critical_alarms()

    # Tier 2: Major - Team notification
    self._create_major_alarms()

    # Tier 3: Minor - Trends
    self._create_minor_alarms()

    # Composite - Overall health
    self._create_composite_alarms()

def _create_critical_alarms(self):
    """Create critical infrastructure alarms"""

    self.cloudwatch.put_metric_alarm(
        AlarmName='DatabaseDown',
        MetricName='HealthyHostCount',
```

```
            Namespace='AWS/RDS',
            Statistic='Minimum',
            Period=60,
            EvaluationPeriods=1,
            Threshold=1,
            ComparisonOperator='LessThanThreshold',
            TreatMissingData='breaching',
            AlarmActions=[
                'arn:aws:sns:us-east-1:123456789012:critical-incidents'
            ]
        )

    def _create_major_alarms(self):
        """Create major service alarms"""

        alarms = [
            ('HighErrorRate', 'ErrorRate', 2),
            ('SlowResponseTime', 'ResponseTime', 500),
            ('HighDatabaseConnections', 'DatabaseConnections', 450)
        ]

        for alarm_name, metric, threshold in alarms:
            self.cloudwatch.put_metric_alarm(
                AlarmName=alarm_name,
                MetricName=metric,
                Namespace='Microservices',
                Threshold=threshold,
                ComparisonOperator='GreaterThanThreshold',
                EvaluationPeriods=2,
                Period=300,
                AlarmActions=[
                    'arn:aws:sns:us-east-1:123456789012:team-alerts'
                ]
            )

    def _create_minor_alarms(self):
        """Create operational monitoring alarms"""
```

```
        self.cloudwatch.put_metric_alarm(
            AlarmName='HighCPU',
            MetricName='CPUUtilization',
            Namespace='AWS/EC2',
            Threshold=75,
            ComparisonOperator='GreaterThanThreshold',
            EvaluationPeriods=5,
            Period=300,
            AlarmActions=[
                'arn:aws:sns:us-east-1:123456789012:ops-notifications'
            ]
        )

    def _create_composite_alarms(self):
        """Create health indicator composite alarms"""

        self.cloudwatch.put_composite_alarm(
            AlarmName='SystemUnhealthy',
            AlarmRule='(ALARM(HighErrorRate) OR ALARM(DatabaseDown)) AND AL
            ActionsEnabled=True,
            AlarmActions=[
                'arn:aws:sns:us-east-1:123456789012:critical-incidents'
            ]
        )
```

Usage

```
strategy = ComprehensiveAlarmStrategy()
strategy.create_alarm_hierarchy()
```

Question 3: Optimize CloudWatch Costs While Maintaining Observability

Scenario: "Your CloudWatch costs are $500/month and you need to reduce them by 30% without losing critical visibility. Design a cost optimization strategy."

Answer Structure:

Cost Analysis:

Current Monthly Cost Breakdown:

Logs Ingestion: $250 (50GB × $0.50)
Logs Storage: $100 (150GB retained × $0.03)
Custom Metrics: $80 (250 metrics × $0.30)
Detailed Monitoring: $40 (40 resources × $0.01)
Alarms: $20 (250 alarms)
Insights Queries: $10
Total: $500

Optimization Strategy:

1. Log Sampling and Filtering

Before: Log everything

Cost: 50GB ingested/month

After: Smart sampling

Ingest 100% of errors

Ingest 10% of normal requests

Ingest 0% of health checks

Expected cost: 20-25GB/month (60% reduction)

class LogOptimization:
def **init**(self):
self.logs = boto3.client('logs')

```
def implement_sampling(self):
    """Implement intelligent log sampling"""

    # Create subscription filter with Lambda processor
    self.logs.put_subscription_filter(
        logGroupName='/myapp/api',
        filterName='SmartSampler',
        filterPattern='',  # All logs
```

```
        destinationArn='arn:aws:lambda:us-east-1:123456789012:function:LogSamp
    )
```

Lambda function that samples logs

def lambda_handler(event, context):
import json
import base64
import random

```
# Decode CloudWatch Logs data
payload = json.loads(gzip.decompress(
    base64.b64decode(event['awslogs']['data'])
))

sampled_events = []
for log_event in payload['logEvents']:
    message = log_event['message']

    # Always send errors
    if 'ERROR' in message or 'EXCEPTION' in message:
        sampled_events.append(log_event)
    # Sample normal logs at 10%
    elif random.random() < 0.10:
        sampled_events.append(log_event)
    # Skip health checks
    elif 'health-check' not in message:
        sampled_events.append(log_event)

# Forward sampled logs
payload['logEvents'] = sampled_events
return {'statusCode': 200}
```

Estimated savings: 60% on log ingestion = $150/month

2. Reduce Metric Retention

Before: All metrics 15 months

After: Tiered retention

- High-frequency metrics: 3 months

- Low-frequency metrics: 1 month

- Summary metrics: 15 months

class MetricRetentionOptimization:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')

```
def implement_tiered_retention(self):
    """Archive old metrics to S3 automatically"""

    # Use Firehose to export metrics to S3 daily
    # Cost: ~$2/month for Firehose + S3 storage
    # Savings: $30-50/month vs keeping all metrics
    pass
```

3. Consolidate Metrics

Before: 250 individual metrics

After: 50 composite metrics + 10 custom metrics

Savings: 200 metrics × $0.30 = $60/month

Group related metrics

consolidation_map = {
# Instead of: cpu-utilization-1, cpu-utilization-2, ... cpu-utilization-50
# Create: avg-cpu-utilization, max-cpu-utilization (aggregates)
'AverageCPU': 'avg(cpu-utilization-*)',
'PeakCPU': 'max(cpu-utilization-)',*

```
# Instead of: request-time-endpoint-1, request-time-endpoint-2, ...
# Create: p50-response-time, p95-response-time, p99-response-time
'P50ResponseTime': 'pct(response-time-all, 50)',
'P95ResponseTime': 'pct(response-time-all, 95)',
'P99ResponseTime': 'pct(response-time-all, 99)',
```

}

4. Optimize Detailed Monitoring

Before: Detailed monitoring (1-min) for all 40 resources

After: Detailed monitoring only for critical resources

Savings: 30 × $0.01 = $0.30/month (minimal but principle matters)

More important: Replace high-frequency metrics with sampling

ec2 = boto3.client('ec2')

Disable detailed monitoring on non-critical instances

ec2.monitor_instances(
InstanceIds=['i-non-critical-1', 'i-non-critical-2'],
Monitoring={'Enabled': False}
)

Keep detailed monitoring on critical instances

ec2.monitor_instances(
InstanceIds=['i-critical-app-1', 'i-critical-app-2'],
Monitoring={'Enabled': True}
)

5. Consolidate Alarms

Before: 250 individual alarms

After: 80 alarms + 30 composite alarms

Savings: 170 alarms × $0.10 = $17/month

Use composite alarms instead of individual alarms

class AlarmConsolidation:
def consolidate_alarms(self):
"""Replace many alarms with composite alarms"""

```
# Before: 20 CPU alarms (one per instance)
# After: 1 composite alarm "HighAverageCPU"
```

```
cloudwatch = boto3.client('cloudwatch')

# Create single metric for all CPU
cloudwatch.put_metric_alarm(
    AlarmName='HighAverageCPU',
    Metrics=[
        {
            'Id': 'cpu_avg',
            'Expression': 'avg([cpu_metric_1, cpu_metric_2, ...])',
            'ReturnData': True
        }
    ],
    Threshold=75,
    ComparisonOperator='GreaterThanThreshold'
)
```

6. Implement Log Insights Caching

Before: Run same queries repeatedly

After: Cache results and reuse

class LogInsightsCaching:
def cache_common_queries(self):
"""Pre-compute common queries and store as metrics"""

```
# Instead of querying logs every hour for error count
# Create metric filter that runs continuously

logs = boto3.client('logs')

logs.put_metric_filter(
    logGroupName='/myapp/api',
    filterName='ErrorCountMetric',
    filterPattern='[msg1, msg2, msg3, status = "ERROR", ...]',
    metricTransformations=[{
        'metricName': 'ErrorCount',
        'metricNamespace': 'MyApp',
```

```
            'metricValue': '1'
        }]
    )

    # Now use metric instead of Insights query
    # Cost: $0 vs $0.005 per Insights query
```

Optimization Summary:

Cost Reduction Plan:

Action | Current | New | Savings
Log Sampling (60% reduction) | $250 | $100| $150
Metric Consolidation | $80 | $30 | $50
Alarm Consolidation | $20 | $15 | $5
Insights Query Reduction | $10 | $2 | $8
Detailed Monitoring Adjustment | $40 | $28 | $12
Total: $225 (45% reduction)

Final Cost: $275/month (vs $500)
Achieved: 45% reduction (exceeds 30% goal)

Remaining Costs Post-Optimization:

{
"logs_ingestion": 100,
"logs_storage": 75,
"custom_metrics": 30,
"detailed_monitoring": 28,
"alarms": 15,
"insights_queries": 2,
"other": 25,
"total": 275
}

Question 4: Troubleshoot CloudWatch Issues and Debug Monitoring Blind Spots

Scenario: "Your alarms didn't trigger during an outage, and you didn't notice it until customers reported issues. Walk through your troubleshooting methodology and what you would implement to prevent this."

Answer Structure:

Investigation Methodology:

Alarm Failure During Outage
↓
Step 1: Verify Alarm Configuration
├─ Is alarm enabled?

├── Are metric dimensions correct?
├── Is threshold appropriate?
└── Are actions configured?
↓
Step 2: Check Metric Data
├── Is metric being collected?
├── Are data points arriving?
├── Check for gaps in data
└── Verify metric namespace/name
↓
Step 3: Analyze Alarm State History
├── When did alarm state change?
├── How many evaluation periods triggered?
├── Compare to actual incident time
└── Look for state transitions
↓
Step 4: Review Action Execution
├── Did SNS publish succeed?
├── Did Lambda invoke?
├── Check CloudTrail for API calls
├── Review action error logs
└── Check target service status
↓
Step 5: Identify Root Cause
├── Configuration error?
├── Metric collection failure?
├── Network/service issue?
├── Threshold too high?
└── Data collection stopped?

Troubleshooting Commands:

Check alarm configuration

aws cloudwatch describe-alarms
--alarm-names "HighCPUUtilization"
--query 'MetricAlarms[0]' | jq .

Get alarm history

aws cloudwatch describe-alarm-history
--alarm-name "HighCPUUtilization"
--max-records 10 | jq .

Check metric data points

aws cloudwatch get-metric-statistics
--namespace "AWS/EC2"
--metric-name "CPUUtilization"
--start-time 2024-01-15T08:00:00Z
--end-time 2024-01-15T10:00:00Z
--period 300
--statistics Average

List metrics for resource

aws cloudwatch list-metrics
--namespace "AWS/EC2"
--dimensions Name=InstanceId,Value=i-1234567890abcdef0

Root Cause Analysis (Common Issues):

Common Alarm Failures:

Issue 1: TreatMissingData Setting
├─ Problem: Alarm set to "notBreaching" when no data
├─ Result: Service crashes, no data collected, no alarm
├─ Solution: Set to "breaching" for critical metrics
└─ Prevention: Use "breaching" as default for critical alarms

Issue 2: Threshold Too High
├─ Problem: Alarm threshold of 95% CPU
├─ Result: Service degrades at 90% but no alarm triggers
├─ Solution: Lower threshold to 80% warning, 90% critical
└─ Prevention: Use SLO-based thresholds (70-75% warning)

Issue 3: Evaluation Periods Too High
├─ Problem: EvaluationPeriods=5, Period=300 = 25 minutes
├─ Result: 25 minutes delay before alarm triggers
├─ Solution: Set to 2-3 periods for faster detection
└─ Prevention: Calculate max acceptable delay

Issue 4: Metric Not Collected
├─ Problem: Custom metric stopped being published
├─ Result: No data in CloudWatch
├─ Solution: Setup alarm on missing data
└─ Prevention: Monitor metric collection itself

Issue 5: SNS/Action Failure
├─ Problem: Alarm triggered but SNS delivery failed
├─ Result: Team not notified
├─ Solution: Redundant notification paths
└─ Prevention: Test alarm actions regularly

Issue 6: Wrong Dimension/Namespace
├─ Problem: Alarm monitoring wrong instance
├─ Result: Alarm never breaches
└─ Solution: Double-check dimensions match resource

Debugging Script:

import boto3
from datetime import datetime, timedelta
import json

class CloudWatchDebugger:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')
self.logs = boto3.client('logs')

```
def debug_alarm_failure(self, alarm_name, start_time, end_time):
    """Comprehensive alarm failure debugging"""

    debug_report = {
        'alarm_name': alarm_name,
        'investigation_time': datetime.now().isoformat(),
        'findings': {}
    }

    # Step 1: Verify alarm exists and enabled
    debug_report['findings']['alarm_config'] = self._check_alarm_config(alarm_na

    # Step 2: Check metric collection
    debug_report['findings']['metric_collection'] = self._check_metric_data(
        alarm_name, start_time, end_time
    )

    # Step 3: Analyze alarm state history
    debug_report['findings']['state_history'] = self._check_alarm_history(
        alarm_name, start_time, end_time
    )

    # Step 4: Verify action execution
    debug_report['findings']['action_execution'] = self._check_action_execution(
        alarm_name
    )
```

```
    # Step 5: Identify issues
    debug_report['issues'] = self._identify_issues(debug_report['findings'])

    # Step 6: Recommendations
    debug_report['recommendations'] = self._generate_recommendations(
        debug_report['issues']
    )

    return debug_report

def _check_alarm_config(self, alarm_name):
    """Verify alarm configuration"""

    try:
        response = self.cloudwatch.describe_alarms(AlarmNames=[alarm_name])

        if not response['MetricAlarms']:
            return {'status': 'ERROR', 'message': 'Alarm not found'}

        alarm = response['MetricAlarms'][0]

        return {
            'status': 'OK',
            'enabled': alarm['ActionsEnabled'],
            'threshold': alarm['Threshold'],
            'comparison': alarm['ComparisonOperator'],
            'evaluation_periods': alarm['EvaluationPeriods'],
            'period': alarm['Period'],
            'statistic': alarm['Statistic'],
            'treat_missing_data': alarm.get('TreatMissingData', 'notBreaching'),
            'alarm_actions': alarm.get('AlarmActions', [])
        }
    except Exception as e:
        return {'status': 'ERROR', 'message': str(e)}

def _check_metric_data(self, alarm_name, start_time, end_time):
    """Check if metric data was collected"""
```

```
    try:
        alarm = self.cloudwatch.describe_alarms(AlarmNames=[alarm_name])['Me

        # Get metric data
        response = self.cloudwatch.get_metric_statistics(
            Namespace=alarm['Namespace'],
            MetricName=alarm['MetricName'],
            Dimensions=alarm.get('Dimensions', []),
            StartTime=start_time,
            EndTime=end_time,
            Period=alarm['Period'],
            Statistics=[alarm['Statistic']]
        )

        datapoints = response['Datapoints']

        return {
            'status': 'OK' if datapoints else 'NO_DATA',
            'datapoints_collected': len(datapoints),
            'expected_datapoints': int((end_time - start_time).total_seconds() / alarm['
            'data_gap_percentage': 100 - (len(datapoints) * alarm['Period'] / (end_time
            'latest_value': datapoints[-1]['Sum'] if datapoints else None
        }
    except Exception as e:
        return {'status': 'ERROR', 'message': str(e)}

def _check_alarm_history(self, alarm_name, start_time, end_time):
    """Check alarm state transitions"""

    try:
        response = self.cloudwatch.describe_alarm_history(
            AlarmName=alarm_name,
            StartDate=start_time,
            EndDate=end_time,
            MaxRecords=100
        )
```

```
        history = response['AlarmHistoryItems']

        return {
            'status': 'OK',
            'state_changes': len(history),
            'transitions': [
                {
                    'timestamp': item['Timestamp'].isoformat(),
                    'new_state': item['HistoryData']
                }
                for item in history[:10]
            ]
        }
    except Exception as e:
        return {'status': 'ERROR', 'message': str(e)}

def _check_action_execution(self, alarm_name):
    """Verify alarm actions executed"""

    # Check CloudTrail for alarm action invocations
    # This would require CloudTrail access and logs

    return {
        'status': 'MANUAL_CHECK_REQUIRED',
        'note': 'Check CloudTrail logs and target service logs (SNS, Lambda, etc.)',
        'check_points': [
            'SNS topic subscription and delivery',
            'Lambda function permissions',
            'AutoScaling action eligibility',
            'Service health at time of alarm'
        ]
    }

def _identify_issues(self, findings):
    """Identify problems from findings"""

    issues = []
```

```
    # Issue: No metric data
    if findings['metric_collection']['status'] == 'NO_DATA':
        issues.append({
            'severity': 'CRITICAL',
            'issue': 'Metric collection failed',
            'impact': 'Alarm cannot trigger without data'
        })

    # Issue: High data gap
    if findings['metric_collection']['data_gap_percentage'] > 20:
        issues.append({
            'severity': 'HIGH',
            'issue': f"Data gap of {findings['metric_collection']['data_gap_percentage']}
            'impact': 'Alarm may not trigger during outage'
        })

    # Issue: Alarm disabled
    if not findings['alarm_config']['enabled']:
        issues.append({
            'severity': 'CRITICAL',
            'issue': 'Alarm actions disabled',
            'impact': 'Notifications will not be sent'
        })

    # Issue: Missing data handling
    if findings['alarm_config']['treat_missing_data'] == 'notBreaching':
        issues.append({
            'severity': 'HIGH',
            'issue': 'TreatMissingData set to notBreaching',
            'impact': 'Alarm will not trigger if metric stops being published'
        })

    return issues

def _generate_recommendations(self, issues):
    """Generate fix recommendations"""

    recommendations = []
```

```
for issue in issues:
    if issue['issue'] == 'Metric collection failed':
        recommendations.append({
            'action': 'Verify metric collection',
            'steps': [
                'Check application is publishing metrics',
                'Verify IAM permissions for CloudWatch API',
                'Check network connectivity to CloudWatch',
                'Review application logs for errors'
            ]
        })

    elif 'Data gap' in issue['issue']:
        recommendations.append({
            'action': 'Investigate data collection gaps',
            'steps': [
                'Check application uptime',
                'Review CloudWatch agent status',
                'Look for service disruptions'
            ]
        })

    elif 'Alarm actions disabled' in issue['issue']:
        recommendations.append({
            'action': 'Enable alarm actions',
            'cli': f'aws cloudwatch enable-alarm-actions --alarm-names <alarm-nam
        })

    elif 'TreatMissingData' in issue['issue']:
        recommendations.append({
            'action': 'Change TreatMissingData to "breaching"',
            'rationale': 'Critical alarms should alert when data stops',
            'cli': '''
            aws cloudwatch put-metric-alarm \\
              --alarm-name <alarm> \\
              --treat-missing-data breaching
            '''
```

```
            })

    return recommendations
```

Usage

debugger = CloudWatchDebugger()

start = datetime.now() - timedelta(hours=2)
end = datetime.now()

report = debugger.debug_alarm_failure(
'HighCPUUtilization',
start,
end
)

print(json.dumps(report, indent=2, default=str))

Prevention Strategies:

class PreventiveMonitoring:
def **init**(self):
self.cloudwatch = boto3.client('cloudwatch')

```
def setup_monitoring_of_monitoring(self):
    """Monitor the monitoring system itself"""

    # 1. Alarm for missing metric data
    self.cloudwatch.put_metric_alarm(
        AlarmName='MetricCollectionFailure',
        Metrics=[
            {
                'Id': 'm1',
                'ReturnData': True,
                'MetricStat': {
                    'Metric': {
                        'Namespace': 'AWS/EC2',
                        'MetricName': 'CPUUtilization'
                    },
                    'Period': 300,
                    'Stat': 'SampleCount'
                }
```

```
            }
        ],
        Threshold=0,
        ComparisonOperator='LessThanOrEqualToThreshold',
        EvaluationPeriods=2,
        TreatMissingData='breaching'
    )

    # 2. Daily alarm audit
    self.cloudwatch.put_metric_alarm(
        AlarmName='DisabledAlarmDetection',
        # Custom metric that tracks disabled alarms
        MetricName='DisabledAlarmCount',
        Namespace='CloudWatchHealth',
        Threshold=0,
        ComparisonOperator='GreaterThanThreshold'
    )

    # 3. Test alarms regularly
    self._schedule_alarm_tests()

def _schedule_alarm_tests(self):
    """Schedule regular alarm tests"""

    # Use EventBridge to trigger Lambda that:
    # 1. Publishes test metric
    # 2. Verifies alarm triggers
    # 3. Confirms notification received

    eventbridge = boto3.client('events')

    eventbridge.put_rule(
        Name='DailyAlarmTest',
        ScheduleExpression='cron(0 3 * * ? *)',  # 3 AM daily
        State='ENABLED'
    )

    eventbridge.put_targets(
```

```
        Rule='DailyAlarmTest',
        Targets=[{
            'Arn': 'arn:aws:lambda:us-east-1:123456789012:function:AlarmTester',
            'Id': '1'
        }]
    )
```

Question 5: Design Multi-Team Monitoring Architecture for Large Organization

Scenario: "Design a monitoring architecture for a large organization with multiple teams, applications, and environments. Include dashboard strategy, alarm delegation, and cost management across teams."

Answer Structure:

Multi-Team Monitoring Architecture:

Organization Structure:

```
Executive/Platform
├─ Global Dashboards (system-wide health)
├─ Billing Dashboard (cost tracking)
├─ SLA Dashboard (compliance)
└─ Alert Routing/Escalation

Team Leads (Per Team)
├─ Team-specific Dashboards
├─ Resource Isolation
└─ Alert Management

Engineers
├─ Application Dashboards
├─ Logs & Metrics Access
└─ On-call Alerts
```

Implementation Strategy:

```
class MultiTeamMonitoringArchitecture:
def init(self):
self.cloudwatch = boto3.client('cloudwatch')
self.logs = boto3.client('logs')
self.iam = boto3.client('iam')
```

```
    def setup_team_namespaces(self):
        """Create isolated namespaces per team"""

        teams = ['payment', 'inventory', 'shipping', 'customer']
```

```
    for team in teams:
        # Custom namespace for team
        namespace = f'Organization/Teams/{team}'

        # Team-specific log groups
        log_group = f'/team/{team}/application'

        try:
            self.logs.create_log_group(logGroupName=log_group)
            self.logs.tag_log_group(
                logGroupName=log_group,
                tags={
                    'Team': team,
                    'Environment': 'production',
                    'CostCenter': f'cc-{team}'
                }
            )
        except:
            pass

def setup_team_dashboards(self):
    """Create team-specific dashboards"""

    teams_config = {
        'payment': {
            'metrics': [
                'PaymentProcessed',
                'PaymentError',
                'PaymentLatency'
            ],
            'resources': ['payment-api', 'payment-db'],
            'audience': 'Payment Team'
        },
        'inventory': {
            'metrics': [
                'InventoryUpdated',
                'StockCheck',
```

```
                'OutOfStock'
            ],
            'resources': ['inventory-service', 'inventory-db'],
            'audience': 'Inventory Team'
        }
    }

    for team, config in teams_config.items():
        self._create_team_dashboard(team, config)

def _create_team_dashboard(self, team, config):
    """Create dashboard for specific team"""

    widgets = []

    # Add metric widgets
    for metric in config['metrics']:
        widgets.append({
            'type': 'metric',
            'properties': {
                'metrics': [[f'Organization/Teams/{team}', metric]],
                'period': 300,
                'stat': 'Average',
                'title': metric
            }
        })

    # Add log table
    widgets.append({
        'type': 'log',
        'properties': {
            'query': f'''
                fields @timestamp, @message, @logStream
                | filter @logStream like /{team}/
                | stats count() by bin(5m)
            ''',
            'title': 'Log Activity'
        }
```

```
    })

    dashboard_body = json.dumps({'widgets': widgets})

    self.cloudwatch.put_dashboard(
        DashboardName=f'{team}-team-dashboard',
        DashboardBody=dashboard_body
    )

def setup_rbac_for_dashboards(self):
    """Setup role-based access to dashboards"""

    teams = ['payment', 'inventory', 'shipping']

    for team in teams:
        # Create team role
        role_name = f'{team}-team-role'

        try:
            self.iam.create_role(
                RoleName=role_name,
                AssumeRolePolicyDocument=json.dumps({
                    'Version': '2012-10-17',
                    'Statement': [{
                        'Effect': 'Allow',
                        'Principal': {
                            'AWS': f'arn:aws:iam::ACCOUNT:group/{team}-team'
                        },
                        'Action': 'sts:AssumeRole'
                    }]
                })
            )

            # Attach team-specific policy
            policy = {
                'Version': '2012-10-17',
                'Statement': [
                    {
```

```
                    'Effect': 'Allow',
                    'Action': [
                        'cloudwatch:GetDashboard',
                        'cloudwatch:GetMetricStatistics',
                        'logs:FilterLogEvents'
                    ],
                    'Resource': [
                        f'arn:aws:cloudwatch:*:*:dashboard/{team}-*',
                        f'arn:aws:logs:*:*:log-group:/team/{team}/*'
                    ]
                }
            ]
        }

        self.iam.put_role_policy(
            RoleName=role_name,
            PolicyName=f'{team}-cloudwatch-policy',
            PolicyDocument=json.dumps(policy)
        )
    except:
        pass

def setup_alert_routing(self):
    """Setup intelligent alert routing per team"""

    eventbridge = boto3.client('events')
    sns = boto3.client('sns')

    # Create SNS topics for each team
    teams = ['payment', 'inventory', 'shipping']

    for team in teams:
        # Team-specific SNS topic
        topic = sns.create_topic(Name=f'{team}-alerts')
        topic_arn = topic['TopicArn']

        # Subscribe team to topic
        sns.subscribe(
```

```
            TopicArn=topic_arn,
            Protocol='email',
            Endpoint=f'{team}-team@company.com'
        )

        # Route alarms for this team to its topic
        eventbridge.put_rule(
            Name=f'{team}-alarm-routing',
            EventPattern=json.dumps({
                'source': ['aws.cloudwatch'],
                'detail-type': ['CloudWatch Alarm State Change'],
                'detail': {
                    'alarmName': [{
                        'prefix': team
                    }]
                }
            }),
            State='ENABLED'
        )

        eventbridge.put_targets(
            Rule=f'{team}-alarm-routing',
            Targets=[{
                'Arn': topic_arn,
                'Id': '1'
            }]
        )

def setup_cost_tracking(self):
    """Setup per-team cost tracking"""

    # Use tags to track costs
    tag_strategy = {
        'Team': 'payment|inventory|shipping|platform',
        'Environment': 'production|staging|development',
        'CostCenter': 'cc-xxxx',
        'Owner': 'team-lead-name'
    }
```

```
# CloudWatch Logs already tagged
# Use AWS Cost Explorer to report costs by tag

# Create custom metric for cost tracking
ce = boto3.client('ce')

# Query costs by tag
response = ce.get_cost_and_usage(
    TimePeriod={
        'Start': '2024-01-01',
        'End': '2024-01-31'
    },
    Granularity='MONTHLY',
    Metrics=['UnblendedCost'],
    GroupBy=[
        {
            'Type': 'TAG',
            'Key': 'Team'
        }
    ],
    Filter={
        'Dimensions': {
            'Key': 'SERVICE',
            'Values': ['Amazon CloudWatch']
        }
    }
)

# Report costs
for result in response['ResultsByTime']:
    for group in result['Groups']:
        team = group['Keys'][0]
        cost = float(group['Metrics']['UnblendedCost']['Amount'])

        print(f"{team}: ${cost:.2f}")
```

Dashboard Hierarchy:

Level 1: Executive Dashboard (System-wide)
├─ Overall System Health (Composite Alarms)
├─ Key Metrics (Requests/sec, Error Rate, Uptime)
├─ SLA Compliance
└─ Cost Trends

Level 2: Service Lead Dashboard (Team-specific)
├─ Team Resource Health
├─ Team Alarms Status
├─ Error Trends
└─ Performance Metrics

Level 3: Engineer Dashboard (Application-specific)
├─ Real-time Metrics
├─ Recent Logs
├─ Specific Endpoint Status
└─ Detailed Performance Analysis

Cost Allocation Matrix:

CloudWatch Costs by Component:

Metrics:
├─ Team A: 50 metrics × $0.30 = $15/month
├─ Team B: 40 metrics × $0.30 = $12/month
└─ Platform: 60 metrics × $0.30 = $18/month

Logs:
├─ Team A: 15GB ingestion × $0.50 = $7.50/month
├─ Team B: 20GB ingestion × $0.50 = $10/month
└─ Platform: 40GB ingestion × $0.50 = $20/month

Alarms: $50/month (shared infrastructure)

Total: ~$200/month
Allocation:
├─ Team A: 30% = $60
├─ Team B: 35% = $70
└─ Platform: 35% = $70

References

[1] Amazon CloudWatch User Guide - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/

[2] CloudWatch Metrics Concepts - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch_concepts.html

[3] CloudWatch Logs User Guide - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/

[4] CloudWatch Logs Insights Query Syntax - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CWL_QuerySyntax.html

[5] CloudWatch Alarms Best Practices - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Best-Practice-Alarms.html

[6] CloudWatch Dashboards - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch_dashboards.html

[7] Using EventBridge with CloudWatch - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch-and-eventbridge.html

[8] A Comprehensive Guide to AWS CloudWatch Monitoring - Economize. https://www.economize.cloud/blog/cloudwatch-explained/

[9] AWS Monitoring Best Practices - SignOz. https://signoz.io/guides/aws-monitoring/

[10] How to Cut Down Amazon CloudWatch Costs - Last9. https://last9.io/blog/how-to-cut-down-aws-cloudwatch-costs/

[11] CloudWatch Metrics Insights - AWS Documentation. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Create_Metrics_Insights_Alarm.html

[12] Best Practices for Monitoring with AWS and OpenTelemetry - Sawmills. https://www.sawmills.ai/blog/best-practices-for-monitoring-with-aws-and-opentelemetry

[Content inserted from previous search - use above full markdown content]